# Overview+data gathering

Updated dataset:
origin_zone,dest_zone,road_type,speed_limit_kmh,num_lanes,is_one_way,distance_km,road_curvature,has_signal,is_construction,weather_condition,day_of_week,time_slot,is_weekend,is_holiday,day_type,congestion_multiplier

train rw model (uses: road_type, weather, day_of_week, is_holiday, traffic)
train fuel model (uses: distance, traffic, time, fuel)

Stages:

1) Curation of dataset

4 forms of data needed:

a) Historical traffic context
- Manually as residents divided Lahore into zones and time slots.

b) Road conditions
- osmnx

c) Environment factors
- Weather getting simulated, can later be replaced with weather API,
- Date time from system, is holiday, is weekend from system

d) Base time
- Precomputed using full algo: `return {`

```
            'base_time':       total_time,
            'distance_km':     max(total_dist / 1000.0, 0.01),
            'road_type':       dominant_rt,
            'has_signal':      '1' if signal_count > 0 else '0',
            'is_one_way':      str(one_way),
            'speed_limit_kmh': sum(speeds) / n,
            'num_lanes':       sum(lanes_list) / n,
            'road_curvature':  curvature,
        }
```

-

**Start + end:** google maps link ->extract lat long from it
**distance :** Use OpenStreetMap + osmnx
For every pair of points: distance = shortest_path_length(graph, A, B)
This gives:realistic road distance (not straight-line)
**Traffic:** Simulated smartly// can be broken down into further sections, manually encoding weights for traffic hours to simulate real data

```
if peak_hours:
    traffic = random.uniform(1.5, 2.5)
else:
```

```
    traffic = random.uniform(1.0, 1.5)
```

**Time:**time = distance / avg_speed

```
if traffic == "high":
    time *= 1.8
elif traffic == "medium":
    time *= 1.3
```

**Fuel:** this is the target variable

fuel = (0.05 * distance) + (0.02 * load) + (0.5 * traffic)

**Vehicle Efficiency for fuel estimation:**

```
if vehicle == "bike":
    efficiency = 40
elif vehicle == "car":
    efficiency = 15
```

2) GA ->In what order should I visit all these locations?

- 👉 Output:
- ✔ The *best order of stops* (not the roads, just the sequence)

3) Navigation Layer (Traffic-Aware Google Maps layer) **Question it answers:**

   👉 *"What exact roads should I take between each stop?"* For each pair (A→C, C→B, etc.), : Finds the **best path on the road network**

4) **Prediction Layer (Linear Regression)**

**Question it answers:**
👉 *"How much fuel will this route consume?"*

- Takes inputs like:
  - Total distance (from GA + A*)
  - Traffic delay (from A*)
  - Cargo weight
- Uses a **Linear Regression model** to estimate:
  - Fuel consumption (liters)
  - Possibly cost

GA

# Genetic Algorithm

**Initial information:**

- All lat longs retracted from google maps links given by user
- If any package is time sensitive, when it should be delivered
- Starting time of deliveries and starting point

**Predetermined values:**

- Mutation rate
- Wd, Wt, Wp
- Population size
- When to stop

**Chromosome configuration :**

[Start, A, C, B, D, End]

**Fitness Function Evaluation:**

cost = Σ normalize((baseTime x mt) + P(u,v))  for all segments in chromosome

- **1. Basetime — base travel time from road conditions + osmnx (Base Speed)**

- **2.mt — manually adjusted time-of-day traffic weight** (Dynamic Traffic)

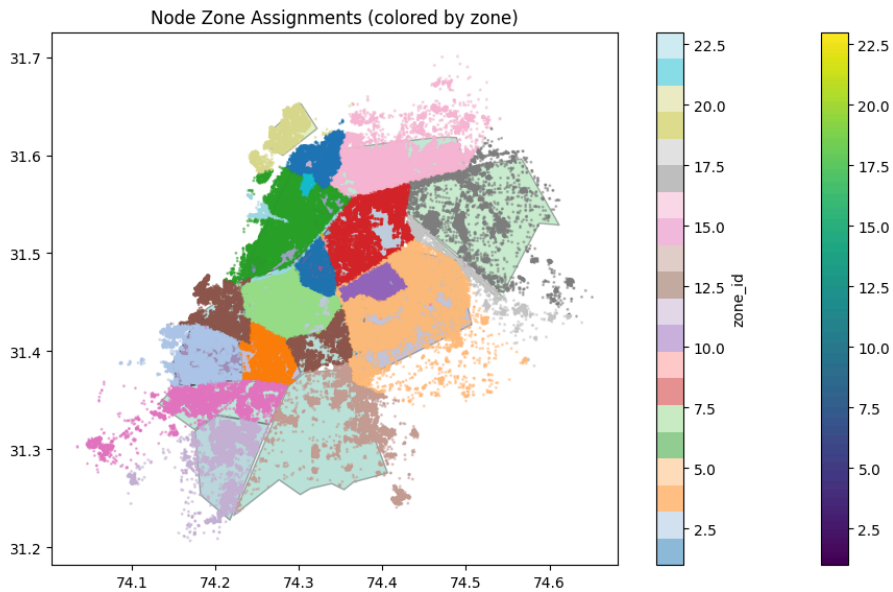
```
CAT_FEATURES = [

    "origin_zone","dest_zone","road_type","is_one_way","has_signal",

    "is_construction","weather_condition","day_of_week","time_slot",

    "is_weekend","is_holiday","day_type",

]

NUM_FEATURES =
["speed_limit_kmh","num_lanes","distance_km","road_curvature"]

ALL_FEATURES  = CAT_FEATURES + NUM_FEATURES
```

rw(u,v) = model.predict(CAT_FEATURES)

*Zone classification :* Lahore divided into custom zones drawn as polygons. Every OSMnx edge classified at graph build time by which zone its midpoint falls in. Zone profiles assigned from local knowledge:

Polygons drawn once in geojson.io, points assigned their relevant zones and modified lahore graph with zone assignment to nodes saved in a graphml file to prevent recomputation.

Visual of zone assignmnet in lahore :




*Tier 2 —*

During evaluation, you track a "running clock." which updates times as the GA "travels" through the chromosome (weight gotten using multiplier using model), for every pair of points, every time_slot also cached to prevent run time computations.

```
TIME_SLOTS = {

    'early_morning': (0,  7),   # 12am–7am  — near empty

    'morning_peak':  (7,  10),  # 7am–10am  — school + office rush

    'midday':        (10, 14),  # 10am–2pm  — moderate

    'afternoon':     (14, 17),  # 2pm–5pm   — school pickup, building

    'evening_peak':  (17, 21),  # 5pm–9pm   — worst slot in Lahore

    'night':         (21, 24),  # 9pm–12am  — winding down

}
```

- **P – penalties**

  Penalty = Lateness^2 x 0.01

This makes a 10-minute delay feel like nothing (1 point), but a 120-minute delay feels massive (144 points). This keeps the "30-minute late" routes alive while naturally filtering out the "4-hour late" disasters.

**Lateness: max(0, ArrivalTime - Deadline)**

The running clock variable also updates traffic weight. If the clock exceeds a parcel's deadline when the GA "visits" that stop, you add a massive penalty to the cost.

> **Example:** If Parcel C must be delivered by 10:00 AM, and your simulation reaches C at 10:30 AM, you add **some number** to the cost. This makes that specific sequence "unfit," causing the GA to naturally evolve toward sequences where C appears earlier

**Crossover:**
Ordered Crossover (OX) or Partially Mapped Crossover (PMX).
**Mutation:**
Adaptive mutation

| Phase | Mutation Rate | Why? |
|---|---|---|
| **Early** | **High (e.g., 0.1)** | To explore the entire "map" and find diverse potential routes. |
| **Mid** | **Medium ( 0.03)** | To start narrowing down on the most promising sequences. |
| **Late** | **Low (e.g., 0.005)** | To "fine-tune" the best routes without accidentally breaking them. |

**Mutation Method:**
Inversion Mutation

## Why Inversion?

In routing, the *relative order* of stops is what matters.

- **Swap Mutation** (picking two stops and swapping them) is okay, but it breaks the sequence in four places.
- **Inversion Mutation** picks two points in the chromosome and reverses the entire segment between them. This only breaks the sequence in two places, preserving more of the "good" sub-routes the GA has already discovered.

**How it looks:**

1. **Original:** `[Start, A, B, C, D, E, F, End]`
2. **Pick Segment:** `[B, C, D, E]`
3. **Invert:** `[Start, A, E, D, C, B, F, End]`

Elitism also done. carry the top 1 or 2 best chromosomes from the current generation into the next one without mutating them.

Layers of GA:

1) Precomputation layer
   2 lookup tables created, a) For all_stops x all_stops, base time, b) For all_stops x all_stops x all_time_slot traffic multiplier
2) GA lifetime running -> simple lookups from our table with a running clock
3) Top 3 solutions selected

—----------

**Number of solutions taken forward to A* layer:**
**3 best candidates**

hotspot

**Three levels of mt:**

python
mt = tier1 × tier2 × hotspot (if applicable)

| Zone | Primary Areas | Profile Characteristics |
|---|---|---|
| **Zone 1 (South-West Hub)**<br><br>**Residential partially dense** | Valencia, Johar Town, Wapda Town, Township | Wide roads but high signal frequency and school traffic. |
| **Zone 2 (Central Residential)**<br><br>**Residential low density** | Model Town, Garden Town, Faisal town | Residential grids; slow speeds but predictable. |
| **Zone 3 (Commercial/Cantt)**<br><br>**Commercial slow** | Gulberg, Cantt, Walton | High-security checkpoints (Cantt) and heavy commercial flow (Gulberg). |
| **Zone 4 (South-East Rural)**<br><br>**Low traffic** | Bedian, Barki Road | Narrower, two-lane roads; low traffic but lower speed limits. |

| | | |
|---|---|---|
| **Zone 5 (Old City/Androon)**<br><br>**Residential dense** | Walled City, Bhati, Delhi Gate | **The "GA Killer."** High penalty zone. Extremely narrow, bikes/pedestrians. |
| **Zone 6 (Ferozepur Spine)**<br><br>**dense** | Mazang, Rehman Pura, Muslim Town, Ichra | Dense, chaotic, and heavily impacted by the Metro Bus corridor. |
| **Zone 7 (DHA Phase 1-5)** | DHA Phases 1, 2, 3, 4, 5 | High-speed, signal-free corridors. Very "fit" for GA routing. |
| **Zone 8 (DHA Phase 6-9+)** | DHA Phases 6, 7, 8, 9, 11 (Rahbar) | Low density, very fast travel times. |
| **Zone 9 (The North/West)** | Shahdara, Ravi Road, Badami Bagh | Heavy vehicle/truck traffic. Major bottlenecks at the bridge. |
| **Zone 10 (Canal/Thokar)**<br><br>**Residential partially dense** | EME, Bahria Town, Canal Road West | Long-distance travel with specific "U-turn" delays. |
| **Zone 11(Raiwind)**<br><br>**Partially dense** | | |

| | | |
|---|---|---|
| **Zone 12(multan road)**<br><br>**Partially dense industrial + residential** | | |
| **Zone 13 (kamahan) (industrial area, dense)** | | |
| **Zone 14 (suburbs, low density area)** | | |
| **Zone 15(lake city area)residential smooth traffic area** | | |
| **Zone 16 (Fatehgarh)**<br><br>**Industrial dense** | | |
| **Zone 17 barki area** | | |
| **Zone 20 Shahdara** | | |
| **Zone 21 Anarkali** | | |

| | | |
|---|---|---|
| **ROADS** | | |
| **Zone 18 Barki road** | | |
| **Zone 19 Ichhra-kalma ferozepur patch, shanghai road** | | |
| **Zone 22 Yateem Khana chowk** | | |
| **Zone 23 Canal Road, Gulberg, Jail Road, Peco Road** | | |

| | | |
|---|---|---|
| 1 | androon | |
| 2 | Model town | |
| 3 | Sukh chayn | |
| 4 | Valencia | |
| 5 | DHA low density | |
| 6 | Ferozepur spine | |
| 7 | Johar Town, Township | |
| 8 | Gulberg,cantt | |
| 9 | Central Gulberg | |
| 10 | Dha dense | |
| 11 | raiwind | |
| 12 | MultanRd side | |
| 13 | Kamahan (ring road side) | |
| 14 | Suburbs low density | |
| 15 | Lake city | |
| 16 | Baghban pura | |
| 17 | Barki | |
| 18 | Barki Road | |
| 19 | Ichra-Kalma | |
| 20 | Shahdara | |
| 21 | Anarkali | |
| 22 | yateemkhana | |
| 23 | Canal road | |

# traffic mult model training

# System Overview

The model predicts a **congestion multiplier**, not travel time directly.
This multiplier represents how much slower traffic is compared to ideal (free-flow) conditions.

Final travel time is calculated as:

actual_time = base_time × congestion_multiplier

# 1. Data and Features

## Target Variable

- `congestion_multiplier`
- Range: ~0.55 to 3.5

## Feature Groups

**Spatial:**

- origin_zone
- dest_zone

**Road Characteristics:**

- road_type
- speed_limit_kmh
- num_lanes
- distance_km
- road_curvature
- is_one_way
- has_signal
- is_construction

**Temporal:**

- time_slot (most important feature)
- day_type
- day_of_week
- is_weekend
- is_holiday

**Environment:**

- weather_condition

### Notes

- All categorical features are passed directly to CatBoost (no encoding required)
- No target leakage (future-only variables are not used)

## 2. Model Training

Model used:

CatBoostRegressor

Key parameters:

- iterations = 2000
- learning_rate = 0.04
- depth = 8
- loss_function = RMSE
- early stopping enabled

Dataset split:

- Train / Validation / Test

The model learns how different factors (time, weather, road type, etc.) affect congestion.

## 3. Model Output

The model predicts:

congestion_multiplier

Interpretation:

- ~1.0 → normal traffic
- 1.0 → slower than normal
- ~2.0 → twice as slow
- ~3.0 → heavy congestion
- <1.0 → faster than expected (rare)

# 4. Precomputation (Lookup Table)

A lookup table is created to avoid running the model during routing.

Structure:

(origin_zone, dest_zone, day_type, time_slot) → multiplier

Steps:

1. Generate all zone pairs
2. Combine with time slots and day types
3. Run model predictions
4. Weight by weather probabilities
5. Store final multiplier

Saved as:

zone_multiplier_lookup.parquet

This allows fast retrieval during routing.

# 5. Travel Time Calculation

For each road segment:

1. Compute base time:

base_time = distance / speed_limit

2. Get multiplier from lookup:

multiplier = lookup(...)

3. Compute final time:

actual_time = base_time × multiplier

## 6. Routing / GA Evaluation

For each candidate route:

Initialize:

total_time = 0

For each segment:

total_time += segment_time

If deadlines exist:

lateness = max(0, total_time - deadline)

Final cost:

cost = total_time + (lateness × penalty_weight)

## 7. Fitness Normalization

To stabilize the genetic algorithm:

fitness = 1 - (cost - min_cost) / (max_cost - min_cost)

- Best route → fitness close to 1
- Worst route → fitness close to 0

## 8. Key Advantages

- No manual traffic rules required
- Model learns real-world patterns automatically
- Fast inference using lookup table
- Generalizes well across zones and conditions

# 9. System Summary

The system consists of three parts:

1. Road graph → provides base travel time
2. ML model → provides congestion multiplier
3. Routing algorithm → finds optimal path

Combined:

Graph + ML → realistic travel time → optimized routing

A*

# A* Algorithm

-

Now that the Genetic Algorithm (GA) has handed the Best Order (e.g., `Start → C → A → B → End`), the A* Layer takes over to find the specific streets between those points.

WE SUBLET A* WORK TO GOOGLE MAPS API

**Initial information:**

- 3 best solutions from GA layer

**We run google maps api on the 3 best solutions to get the actual route between them**

2 Things happening on this stage

1) Best out of the three solutions selected
2) Route to take between those points also suggested